Revisiting randomized choices in isolation forests

Mocanu Ștefan
Voinescu David-Ioan
Chiruș Mina-Sebastian

Final Project of "Anomaly Detection" course

Facultatea de Matematică și Informatică
Universitatea din București
January 30, 2026

Contents

1 State of the Art

This section examines the internal decision logic of prominent isolation-based algorithms. While these methods share the goal of isolating anomalies through recursive partitioning, they differ fundamentally in their **splitting strategies**—specifically in how they select the splitting dimensions, normal vectors, and cut points. The integration of this part of the algorithm is shared with the proposed model from the paper, FCF, and is described in Section 2.2 and Section 2.3.

Isolation Forest(IF)

It uses axis-parallel splits with fully random selection.

Node steps:

- Randomly select a feature j from the set of available dimensions d.
- Randomly select a split value v from a uniform distribution between the minimum and the maximum values of feature j in the current node (S):

$$v \sim U(min(S_j), max(S_j))$$

- **Branching Condition:** A data point x is assigned to the left child node if $x_j < v$. Otherwise, it is assigned to the right child.

Extended Isolation Forest(EIF)

It uses random hyperplanes to handle anomalies that are not axis-aligned (oblique splits).

Node steps:

- Generate a random normal vector n from a standard normal distribution $\mathcal{N}(0, 1)$ for each dimension.
- Project the data points onto this normal vector.
- Select a random split point p uniformly within the range of the projected values.
- **Branching Condition:** A point x goes to the left child if:

$$(x - p) \cdot n \leq 0$$

SCiForest(SCiF)

It uses guided hyperplanes. It aims to address the issue where random hyperplanes might pass through sparse regions without actually separating clusters. Unlike EIF, the choice of the normal vector is optimized rather than purely random.

Node steps:

- Generate a set of candidate hyperplanes $\{(n_1, p_1), \ldots, (n_k, p_k)\}$ using the EIF approach.
- For each candidate, calculate the **Standard Deviation Gain** ($\Delta\sigma$), which measures the reduction in dispersion:

$$\Delta\sigma = \sigma(S) - \frac{|S_L|}{|S|}\sigma(S_L) - \frac{|S_R|}{|S|}\sigma(S_R)$$

  Where $\sigma(S)$ is the standard deviation of the projected points in the current node, and S_L, S_R are the resulting subsets.
- **Selection:** Alternatively, some implementations use a **Density-based** criterion to find sparse regions:

$$Gain = \frac{1}{\text{Density}(S_L) + \text{Density}(S_R)}$$

- Select the specific hyperplane (n_i, p_i) that maximizes this criterion.

2 Algorithm

2.1 Node Splitting Mechanisms

This subsection details the specific decision process occuring inside a single node for FCF.

Uses Constructed Projections and Optimal Splits. Unlike standard Isolation Forest which picks split points randomly, FCF constructs a projection vector z by combining multiple features and then searches for the specific threshold s that best separates the data (minimizing impurity). Node steps:

- Create a projection vector z of size m (number of data points) initialized to zeros.
- Repeat p times (where p is a hyperparameter for the number of splitting variables):
  - Choose a variable index v uniformly at random from the available columns $[1, n]$.
  - Extract the column vector $y = X[:, v]$.
  - Draw a random coefficient c from a Normal distribution $\mathcal{N}(0, 1)$.
  - Update z by adding the standardized version of feature y: $z := z + c\frac{y-\bar{y}}{\sigma_y}$. This step build a random linear combination of vectors.
- Instead of picking a random cut, iterate through possible values of s in z to find the one that minimizes the cluster impurity. i.e. choose s from z that minimizes:

$$\frac{m_{\text{left}}\sigma_{z_l} + m_{\text{right}}\sigma_{z_r}}{m_{\text{left}} + m_{\text{right}}}$$

- Divide the data X into left set X_l (where $z_i \leq s$) and right set X_r (where $z_i > s$).

The Gain criterion is:

$$G = \frac{n_L}{n}\sigma(v_{z_L}) + \frac{n_R}{n}\sigma(v_{z_R})$$

Where:

1. m: Number of data points (rows) in the current node.
2. n: Number of variables (columns) in the dataset.
3. z: The constructed projection vector (size m).
4. z_l, z_r: The subsets of z values falling into the left and right children.
5. $m_{\text{left}}, m_{\text{right}}$: The number of points in the left and right children.
6. σ_z: The standard deviation of the values in vector z.

2.2 Forest Construction Algorithm

The construction of the Fair-Cut Forest (FCF) follows an ensemble approach similar to the standard Isolation Forest but incorporates the specific optimized splitting mechanism defined in the node training phase.

The algorithm takes as input the dataset $\mathbf{X}$ of size $m \times n$, the number of trees t, the sub-sampling size s, and the number of splitting variables p.

1. **Sub-sampling:** The algorithm iterates t times to build the forest. In each iteration, a subset $\mathbf{X}_s$ is created by selecting s data points uniformly at random without replacement from the original dataset $\mathbf{X}$. This sub-sampling (typically $s = 256$) is critical for maintaining diversity among the trees and preventing overfitting.
2. **Tree Growth:** For each subset $\mathbf{X}_s$, a tree is grown using the `FairCutTree` process (described in Section A).
   - The root node is initialized with the subset $\mathbf{X}_s$ and current depth $d = 0$.
   - The tree recursively partitions the data using the optimized gain criterion until the constraints (max depth or single data point) are met.
3. **Normalization Factor ($c(s)$):** Once the forest is built, the algorithm calculates the expected path length of an unsuccessful search in a Binary Search Tree (BST) for a dataset of size s. This value, denoted as $c(s)$, serves as the baseline for normalization:
   $$c(s) = 2H(s-1) - \frac{2(s-1)}{s}$$
   where $H(i)$ is the harmonic number, estimated as $\ln(i) + 0.5772156649$ (Euler's constant).
4. **Output:** The result is a set $\mathbb{T}$ containing t trained Fair-Cut Trees and the normalization factor $c(s)$.

2.3 Scoring and Anomaly Detection

The scoring phase determines the degree of anomaly for a new query point $\mathbf{x}$. This process involves traversing the trees using the parameters learned during training and aggregating the path lengths.

1. Tree Traversal and Projection Reconstruction

To calculate the path length $h(\mathbf{x})$ in a single tree, the point $\mathbf{x}$ must traverse from the root to a leaf. Because FCF uses composite projections, the splitting value z must be mathematically reconstructed at each node using the stored training parameters.

- **Initialization:** Upon entering a node, the projection value z is initialized to 0.
- **Reconstruction Loop:** The algorithm iterates through the specific set of variables $v \in \{1, \ldots, p\}$ that were selected during the training of that node. For each variable, the projection is updated:
$$z := z + c_v \frac{x_v - \bar{y}_v}{\sigma_{y_v}}$$
Here, x_v is the value of the feature in the query point, while c_v (random coefficient), $\bar{y}_v$ (mean), and σ_{y_v} (standard deviation) are the specific values stored in the node during training. This ensures the point is projected onto the exact same hyperplane used to define the split.
- **Branching Decision:** The reconstructed z is compared to the optimal split threshold s stored in the node:
  - If $z \leq s$, the traversal continues to the Left child.
  - If $z > s$, the traversal continues to the Right child.
- **Termination:** This recursive process continues until a leaf node is reached. The returned value is the depth of the leaf (the number of edges from the root).

The final anomaly score $S(\mathbf{x}, s)$ is derived by averaging the path lengths across the entire forest $\mathbb{T}$.

1. **Average Path Length:** The query point $\mathbf{x}$ is passed through all t trees. The average path length $E(h(\mathbf{x}))$ is calculated as:
$$E(h(\mathbf{x})) = \frac{1}{t} \sum_{i=1}^{t} h_i(\mathbf{x})$$
2. **Score Normalization:** The average path length is normalized by the expected path length $c(s)$ to produce a score between 0 and 1:
$$S(\mathbf{x}, s) = 2^{-\frac{E(h(\mathbf{x}))}{c(s)}}$$

3. **Interpretation:**

   - $S \to 1$**:** The point has a short average path length, indicating it was easily isolated. It is likely an **anomaly**.
   - $S \to 0.5$**:** The point has an average path length, indicating it is likely a normal observation.
   - $S \to 0$**:** The point is deep inside the cluster, indicating it is very normal.

2.4 Complexity Analysis

This section analyzes the computational complexity of the Fair-Cut Forest (FCF) and contrasts it with the standard Isolation Forest (IF) and Extended Isolation Forest (EIF). The primary trade-off proposed by the FCF model is accepting higher computational costs per node in exchange for optimized splits that better isolate clustered anomalies.

2.4.1 Training Time Complexity

The training complexity is determined by the cost of constructing the trees. For a forest of t trees and a sub-sample size of s:

- **Isolation Forest (IF):** The standard IF selects a feature and a split value uniformly at random. This requires $\mathcal{O}(1)$ operations per node (constant time). The total complexity for building the forest is:

$$\mathcal{O}(t \cdot s \log s)$$

- **Fair-Cut Forest (FCF):** In FCF, the splitting process is significantly more involved. At each node containing m data points:

  1. **Projection Construction:** The algorithm iterates p times (the number of splitting variables). In each iteration, it performs vector operations on m points to update $\mathbf{z}$. This costs $\mathcal{O}(m \cdot p)$.
  2. **Split Optimization:** To find the optimal split s, the projection vector $\mathbf{z}$ is typically sorted, costing $\mathcal{O}(m \log m)$.

  Consequently, the total training complexity for FCF is:

$$\mathcal{O}(t \cdot s \cdot (p + \log s) \cdot \log s)$$

  Since p is a hyperparameter often set > 1, FCF is computationally more expensive than IF. However, the paper argues that this cost is justified as it leads to more efficient isolation (shallower trees for anomalies).

2.4.2 Scoring Time Complexity

The scoring complexity refers to the time required to evaluate a new query point $\mathbf{x}$.

- **Isolation Forest (IF):** A point simply traverses the tree by making single-feature comparisons. The cost is proportional to the tree height ($\log s$).

$$\mathcal{O}(t \cdot \log s)$$

- **Fair-Cut Forest (FCF):** As described in Section 2.3, traversing a node requires reconstructing the projection z.
  - At each node in the path, the algorithm must perform p updates (one for each splitting variable stored during training).
  - Therefore, the cost per node is $\mathcal{O}(p)$.

  The total scoring complexity is:

$$\mathcal{O}(t \cdot p \cdot \log s)$$

  This makes prediction slower than IF by a factor of p, but comparable to Extended Isolation Forest (EIF) if p is close to the data dimensionality n.

2.4.3 Space Complexity

The memory requirement is defined by the parameters stored in the trained model.

- **Isolation Forest (IF):** Stores only the split dimension index and split value per node. Space is minimal: $\mathcal{O}(t \cdot s)$.
- **Fair-Cut Forest (FCF):** For each internal node, FCF must store the parameters required to reconstruct the projection. This includes:
  - The indices of the p variables used.
  - The p random coefficients (c_v).
  - The p means ($\bar{y}_v$) and standard deviations (σ_{y_v}).

  Thus, the space complexity scales linearly with p:

$$\mathcal{O}(t \cdot s \cdot p)$$

3 Implementation

3.1 Models defined

- **iForest**: Original Isolation Forest (Liu et al., 2008)
- **EIF**: Extended Isolation Forest (Hariri et al., 2019)
- **SCiForest**: Split-Criterion Isolation Forest (Liu et al., 2010)
- **FCF**: Fair-Cut Forest (Cortes, 2021)
- **RRCF**: Robust Random Cut Forest (Guha et al., 2016)

3.2 Multi-threading

3.2.1 Tree-level Parallelism

```
#pragma omp parallel for schedule(dynamic) num_threads(nthreads)
for (size_t tree = 0; tree < ntrees; tree++) {
    // Build tree independently
    build_tree(tree, sample, model);
}
```

Each tree is built independently, allowing embarrassingly parallel construction.

3.2.2 Prediction Parallelism

```
#pragma omp parallel for schedule(static) num_threads(nthreads)
for (size_t i = 0; i < n_observations; i++) {
    scores[i] = score_observation(X[i], forest);
}
```

Predictions for different observations are computed in parallel.

3.3 Optimizations

The library supports:

- **-march=native**: Compile-time CPU optimization (AVX, AVX2, etc.)
- **Cache-friendly data layouts**: Flat vector storage instead of pointer-based trees
- **Contiguous memory access**: Improves cache locality during tree traversal

3.4 Data Structures

3.4.1 IsoTree: Standard Node Structure

```
typedef struct IsoTree {
    ColType  col_type;            // Numeric, Categorical, or NotUsed
    size_t   col_num;             // Column index for split
    double   num_split;           // Split threshold (numeric)
    std::vector<char> cat_split; // Bitmap for categorical splits
    int      chosen_cat;          // Selected category
    size_t   tree_left;           // Index to left child
    size_t   tree_right;          // Index to right child
    double   pct_tree_left;       // Fraction going left
    double   score;               // Depth/score at node
    double   range_low;           // Range bounds for penalization
    double   range_high;
    double   remainder;           // For distance calculations
} IsoTree;
```

3.4.2 IsoHPlane: Extended/Hyperplane Node

For extended isolation forest with hyperplane splits:

```
typedef struct IsoHPlane {
    std::vector<size_t>   col_num;    // Multiple columns
    std::vector<ColType>  col_type;   // Type per column
    std::vector<double>   coef;       // Linear combination coeffs
    std::vector<double>   mean;       // Standardization means
    std::vector<double>   fill_val;   // Missing value handling
    std::vector<std::vector<double>> cat_coef; // Categorical coeffs

    double   split_point;             // Threshold on combination
    size_t   hplane_left;             // Left child index
    size_t   hplane_right;            // Right child index
    double   score;
    double   range_low, range_high;
    double   remainder;
} IsoHPlane;
```

3.4.3 IsoForest: Complete Model

```
typedef struct IsoForest {
    std::vector<std::vector<IsoTree>> trees;  // Forest storage
    NewCategAction     new_cat_action;    // Handle new categories
```

```
    CategSplit          cat_split_type;    // Categorical strategy
    MissingAction       missing_action;    // Missing value handling
    ScoringMetric       scoring_metric;    // depth, density, etc.
    double              exp_avg_depth;     // Expected isolation depth
    double              exp_avg_sep;       // Expected separation
    size_t              orig_sample_size; // Training sample size
    bool                has_range_penalty;
} IsoForest;
```

3.4.4 ExtIsoForest: Extended Model

```
typedef struct ExtIsoForest {
    std::vector<std::vector<IsoHPlane>> hplanes;  // Hyperplane trees
    // Same metadata as IsoForest...
} ExtIsoForest;
```

3.5 Design Rationale

3.5.1 Flat Vector Storage

Trees are stored as **flat vectors** rather than traditional pointer-based nodes:

- **Cache efficiency**: Contiguous memory layout
- **Serialization**: Easy to save/load models
- **Thread safety**: No pointer aliasing issues
- **Memory locality**: Children accessed by index, not pointer

```
// Tree traversal by index
size_t current = 0;  // Root
while (!is_terminal(tree[current])) {
    if (x[tree[current].col_num] <= tree[current].num_split)
        current = tree[current].tree_left;
    else
        current = tree[current].tree_right;
}
```

3.6 Benchmarks

From the isotree repository benchmarks (100 trees, CovType dataset):

Library	Model	256 samples	1024 samples	10k samples
isotree	orig	0.00161s	0.00631s	0.0848s
isotree	ext	0.00326s	0.0123s	0.168s
eif	orig	0.149s	0.398s	4.99s
eif	ext	0.16s	0.428s	5.06s
H2O	orig	9.33s	11.21s	14.23s
H2O	ext	1.06s	2.07s	17.31s
scikit-learn	orig	8.30s	8.01s	6.89s
solitude	orig	32.61s	34.01s	41.01s

Table 1: Training time comparison on CovType dataset, 100 trees (lower is better)

Key observation: isotree is typically 1–2 orders of magnitude faster than other implementations.